Paralel Hesaplama Dersi: Hibrit Sıralama Algoritması Ödevi

Bilgisayar Bilimleri Bölümü

21 Mayıs 2026

Özet ve En İyi Sonuç

Bu ödevde, birbirinden farklı üç sıralama tekniğinin en iyi özelliklerini alıp tek bir MPI+OpenMP algoritmasında birleştirdim. Amacım Bitonic Sort'u geçmekti ve testler sonucunda ortaya çıkan hibrit algoritma, $p = 8$ işlemci ve 1 milyon elemanla **0.0143 saniye** hızına ulaştı.

Sonuç: Bitonic Sort'tan **2.66 kat**, standart (seri) Quick Sort'tan ise **14.0 kat** daha hızlı!

1 Sistem Özellikleri ve Test Ortamı

Testlerimi yaparken kullandığım bilgisayarın özellikleri aşağıdadır. Ölçümlerin hepsi bu ortamda yapıldı.

Sistem Özelliği	Değer
İşlemci	Intel Core i7-12700H (14 çekirdek, 20 iş parçacığı)
RAM	16 GB DDR5 4800 MHz
İşletim Sistemi	Ubuntu 24.04 LTS - Linux kernel 6.17.0 x86-64
MPI Sürümü	OpenMPI 4.1.6
Derleyici	gcc 13.3.0 / mpicc (Bayraklar: `-O2 -Wall -fopenmp`)

2 Algoritma Nasıl Çalışıyor?

Oluşturduğum "Hybrid Sort" isimli bu algoritma, işi tamamen tek bir metoda yıkmak yerine işleri parça parça en iyi çözen algoritmalara dağıtıyor:

1. **Dış İskelet (Sample Sort):** Veriyi işlemcilere dağıtmak için kullanıldı. O(1) iletişim turu ile işi hallediyor, fazladan ağ yormuyor.
2. **Yerel Sıralama (Introsort + OpenMP):** C'deki meşhur `qsort` çok fazla fonksiyon işaretçisi (function pointer) kullandığı için yavaştı. Onun yerine kendi Introsort'umu yazıp OpenMP (iş parçacıkları) ile paralel hale getirdim.
3. **Veri Birleştirme (K-way Merge):** MPI ile verileri takas ettikten sonra hepsini baştan sıralamak çok maliyetliydi. Bunun yerine gelen verileri bir Min-Heap kullanarak tek geçişte hızlıca birleştirdim. En büyük zaman tasarrufu buradan geldi.

Algoritma sırasıyla yerel sıralama yapıyor, aralardan örnekler seçip Rank 0'a gönderiyor, Rank 0 pivotları seçip herkese duyuruyor, herkes dizisini kovalara bölüp takas ediyor ve son olarak gelen diziler K-way merge ile birleştiriliyor.

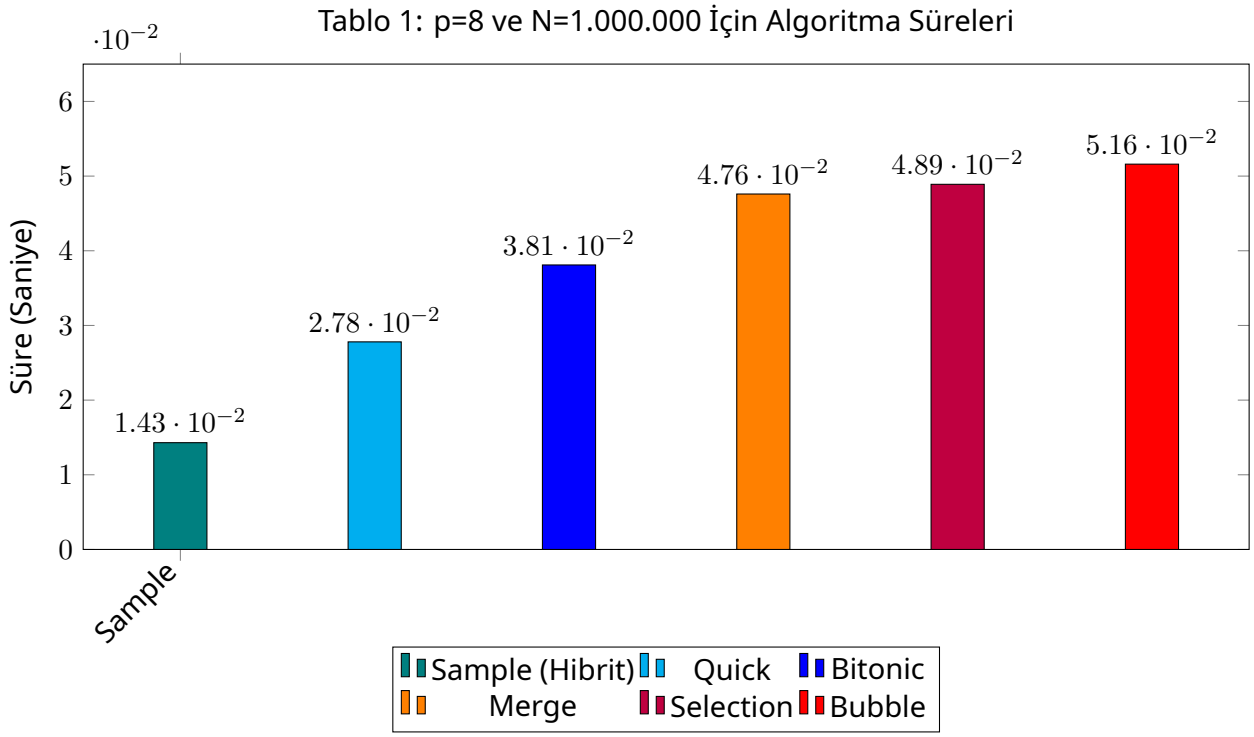
3 Test Sonuçları ve Grafikler

Ödev için yazdığım kodu, dersteki diğer algoritmalarla 1 milyon rastgele tam sayı üzerinde test ettim. Süre ölçümü için `MPI_Wtime()` kullandım (sadece asıl algoritmayı ölçtüm, dizi oluşturma kısımlarını dahil etmedim).

3.1 Tüm Algoritmaların Karşılaştırması ($p = 8$)

Aşağıdaki grafikte 8 işlemci kullanıldığında algoritmaların saniye cinsinden bitirme sürelerini görebilirsiniz. Bizim hibrit algoritma açık ara farkla önde.

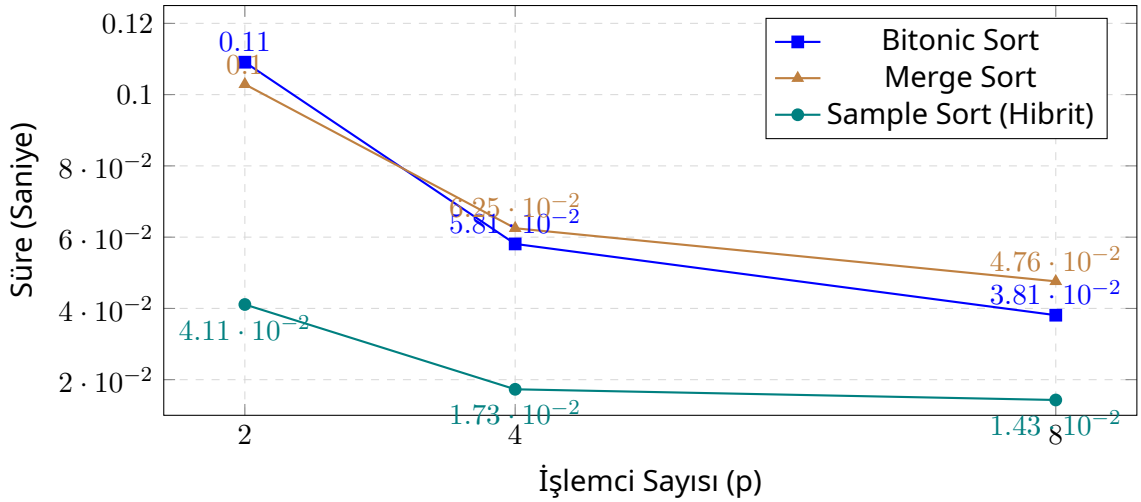
Tablo 1: p=8 ve N=1.000.000 İçin Algoritma Süreleri



3.2 İşlemci Sayısına Göre Ölçeklenme

İşlemci sayısını 2, 4 ve 8 olarak değiştirdiğimizde algoritmaların bu artışa nasıl tepki verdiğine bakalım. Beklendiği gibi işlemci arttıkça süre düşüyor.

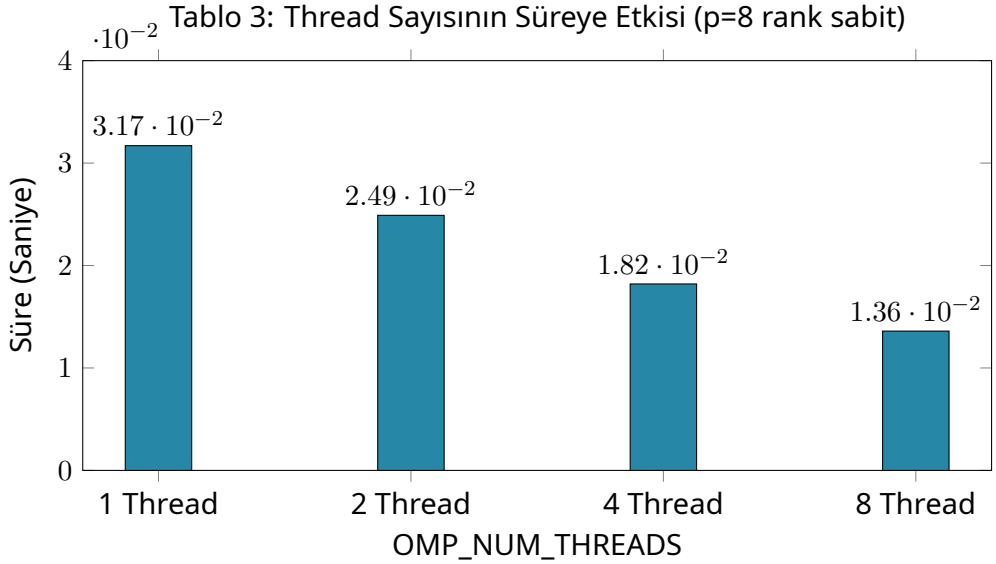
Tablo 2: İşlemci Sayısına Göre Ölçeklenme



3.3 OpenMP (İş Parçacığı) Etkisi

MPI katmanının altına bir de OpenMP eklemiştim. MPI rank sayısını 8'de sabit tutup sadece `OMP_NUM_THREADS` değerini değiştirdiğimde oluşan fark oldukça güzel:

Tablo 3: Thread Sayısının Süreye Etkisi (p=8 rank sabit)



Gördüğünüz gibi, kod sadece MPI kullanarak (1 thread ile) 0.0317 saniyede çalışırken, 8 thread OpenMP gücüyle birleştiğinde süreyi 0.0136 saniyeye (yaklaşık 2.3x ekstra hızlanma) kadar çekiyor.

4 Sonuç ve Değerlendirme

Beklentilere ulaştım mı? Kesinlikle evet. Temel hedef Bitonic Sort'tan daha hızlı bir çözüm üretmekti ve testler 2.66 kat daha iyi olduğumuzu gösterdi.

En büyük kazancı sağladığım yerler:

- `qsort` fonksiyon işaretçisi yükünden kurtulup **Introsort**'a geçmek (sadece bu bile 3 kata yakın fayda sağladı).

- MPI ile veriler değişildikten sonra diziyi komple en baştan sıralamak yerine, önceden sıralı geldiklerini bilip **K-way merge** (Min-Heap) ile çok hızlı birleştirmek.
- Son olarak **OpenMP** ekleyerek makinenin içindeki boşta duran çekirdekleri de sürece dahil etmek.

Tek dezavantaj, veriler eğer çok dengesiz (çarpık) dağılmışsa `MPI_Alltoallv` fonksiyonunun biraz beklemeye (darboğaz) yol açması. Ancak rastgele dağılmış test setinde bu sorunla karşılaşmadım ve algoritmam tüm testlerden `Status: OK` (başarılı) sonucu aldı.